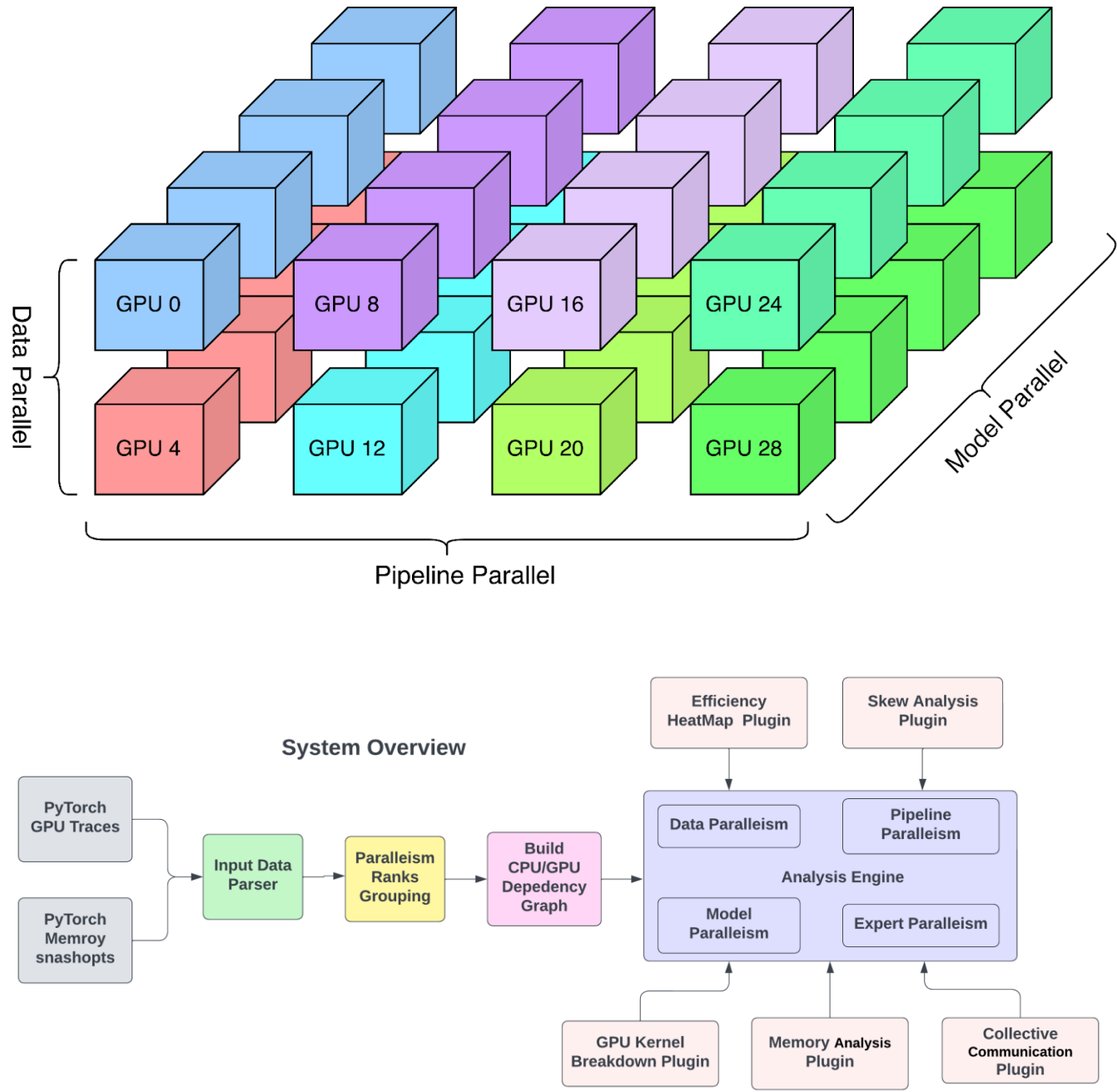


Introduction

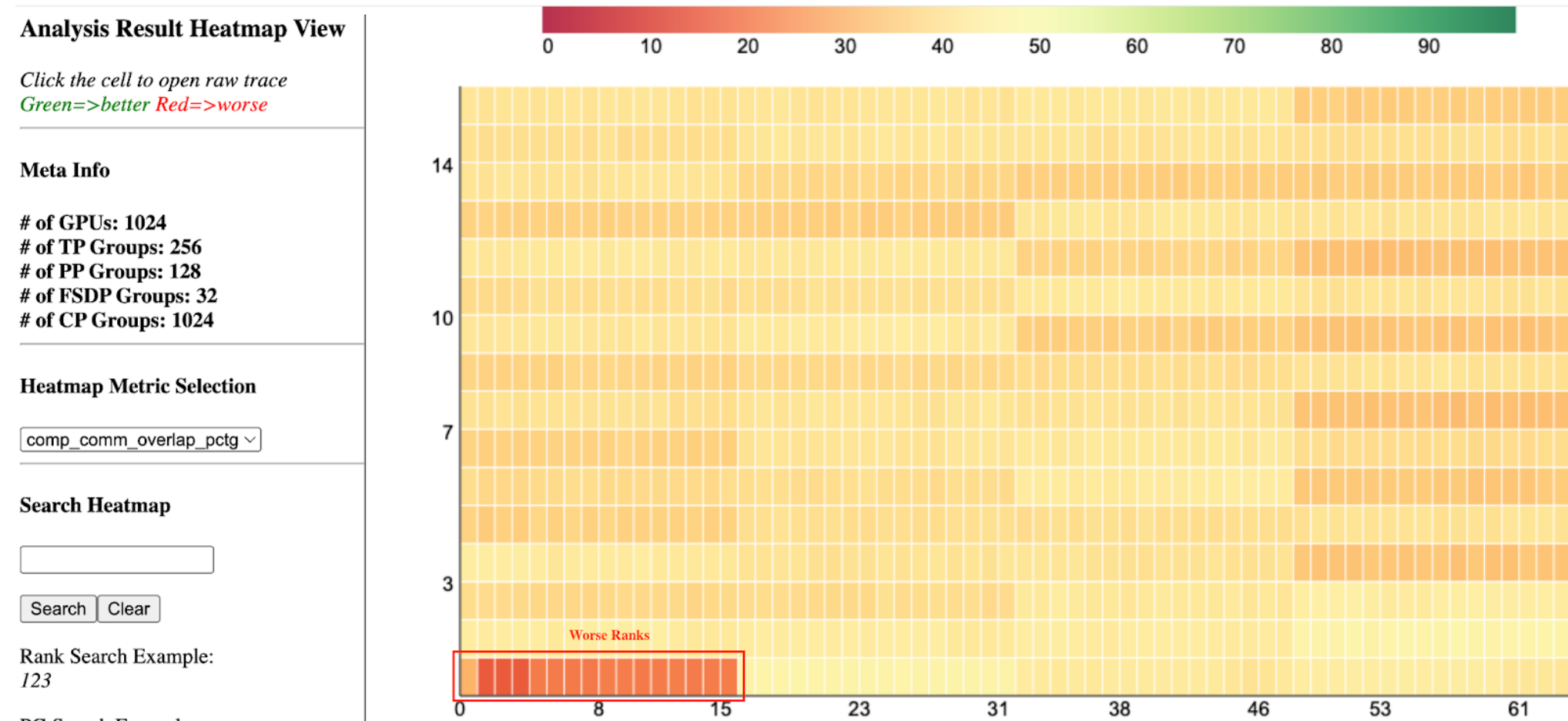
N-dimensional parallelism is a technique that speeds up LLM (Large Language Models) training by distributing the workload across multiple "dimensions" of the model and data, utilizing multiple accelerators to enable faster training and improved memory efficiency. Large-scale distributed training of LLMs on GPU clusters using N-dimensional parallelism has become increasingly common, but analyzing performance bottlenecks and scalability limitations remains a challenge. To address this, new features have been developed to efficiently identify performance bottlenecks and analyze scalability headroom for LLM workloads.

Our system analyzes PyTorch Traces and memory snapshots to provide insights into parallelism, using an analysis engine (data, pipeline, model, etc.) with customizable plugins to offer a comprehensive analysis for debugging N-D parallelism.



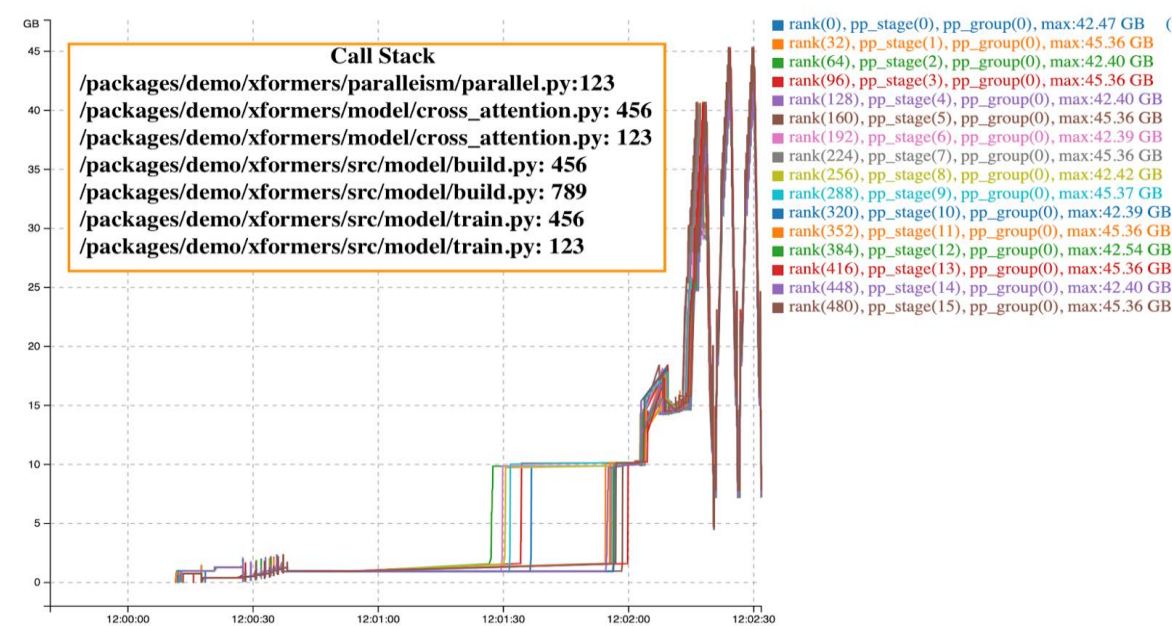
GPU Efficiency Heatmap

The GPU efficiency heatmap provides an overview of GPU runtime efficiency, breaking it down into idle, compute, and non-compute categories. It helps users identify performance trends, spot bottlenecks, and compare efficiency across GPUs or hosts, enabling them to maximize compute time and achieve high training efficiency.



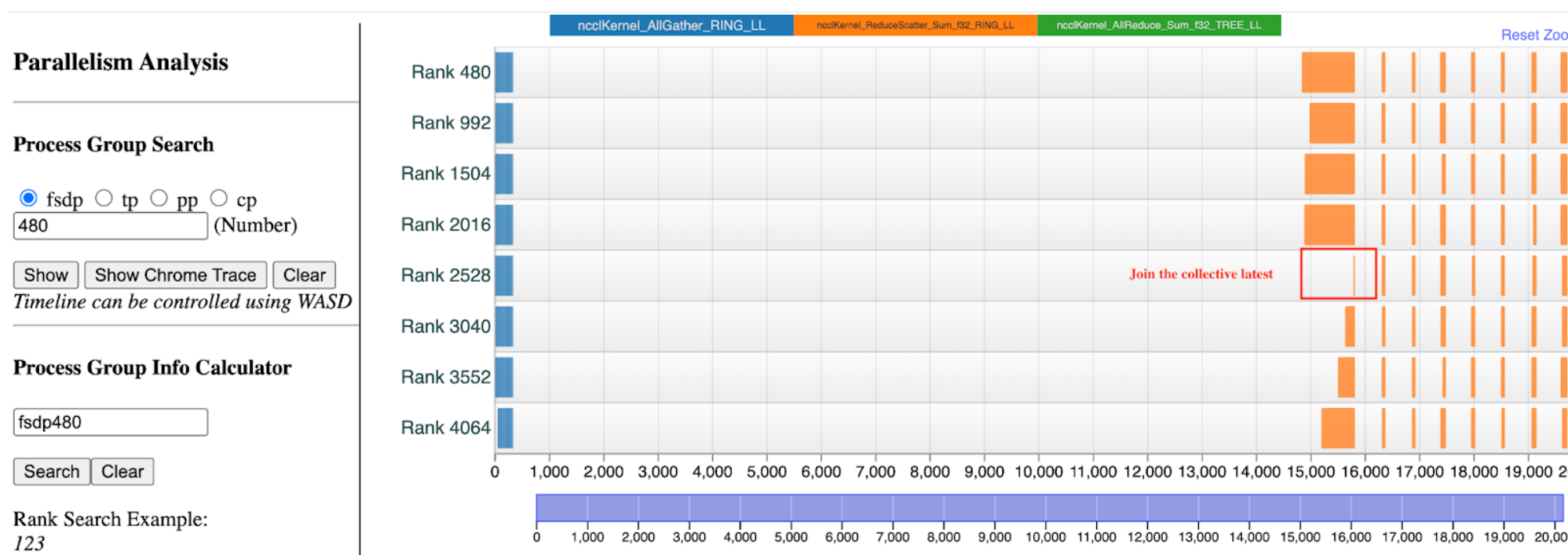
Parallelism-Aware Memory Analysis

The parallelism-aware memory analysis is to identify and diagnose memory imbalance issues in pipeline parallelism, tracing the specific memory allocations that contribute to peak memory usage, providing call stacks to aid in debugging. The visualization indicates that the maximum memory usage reached 45 GB during the observed period.



Parallelism-Aware Trace Analysis

LLM training is distributed across tens of thousands of GPUs, requiring periodic communication to ensure fine-grained parallelism, and balanced load distribution is crucial to optimize training performance, as slow ranks can bottleneck the entire process. This feature groups ranks of the same parallelism type together; extracts collectives associated with the target parallelism and presents a holistic view to help identify stragglers.



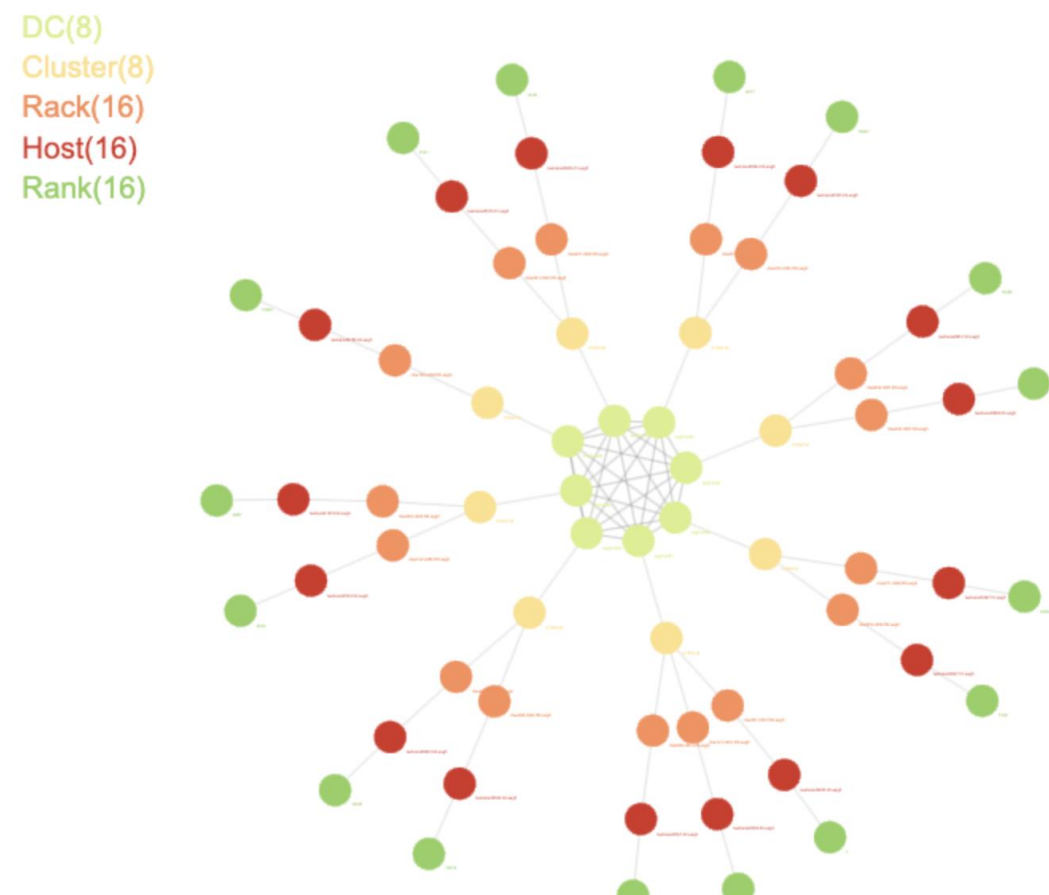
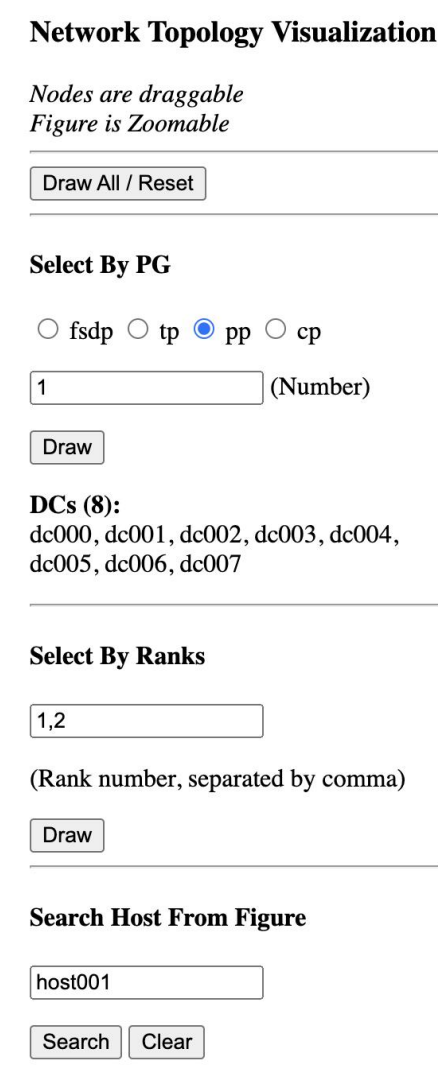
Parallelism-Aware Skew Analysis

Ideally, all ranks should spend similar time on collective operations for maximum parallelism. However, some ranks (rank 2528 as an example in above figure) lagged, causing others to wait. A skew analysis table helps reduce manual effort by providing statistics on collective events, revealing a 97.46% difference in the 8th collective, indicating potential load imbalance in the FSDP group.

Sequence ID	NCCL Name	Earliest Start TS (ms offset)	Earliest Start Rank	Parallelism Group	Latest Start TS (ms offset)	Latest Start Rank	Difference
8	ncclKernel_ReduceScatter_Sum_F32_RING_LL	14832.602626953125	480	fsdp_480	15782.241671875	2528	97.46%
0	ncclKernel_AllGather_RING_LL	0.00069677734375	3552	fsdp_480	50.63673388671875	4064	60.33%
16	ncclKernel_AllReduce_Sum_F32_TREE_LL	20165.349525878908	2016	fsdp_480	20165.847855957032	4064	43.87%
11	ncclKernel_ReduceScatter_Sum_F32_RING_LL	17371.489330078126	992	fsdp_480	17417.533188964844	3552	41.73%

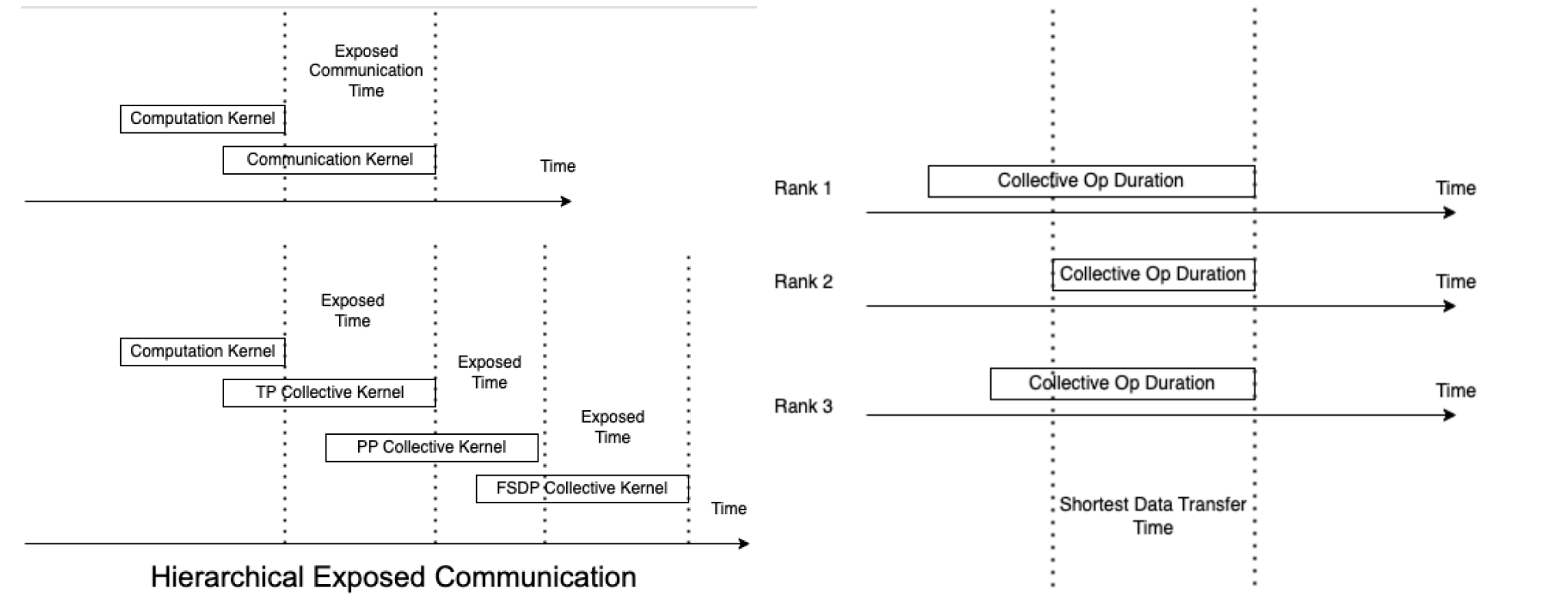
Parallelism-Aware Topology Analysis

This feature visualizes the connections between nodes within the same parallelism process group (below an example for one pipeline parallelism group), allowing for easy detection of performance impacts caused by allocated machines spanning multiple AI Zones, which can lead to increased cross-zone communication degrading performance.



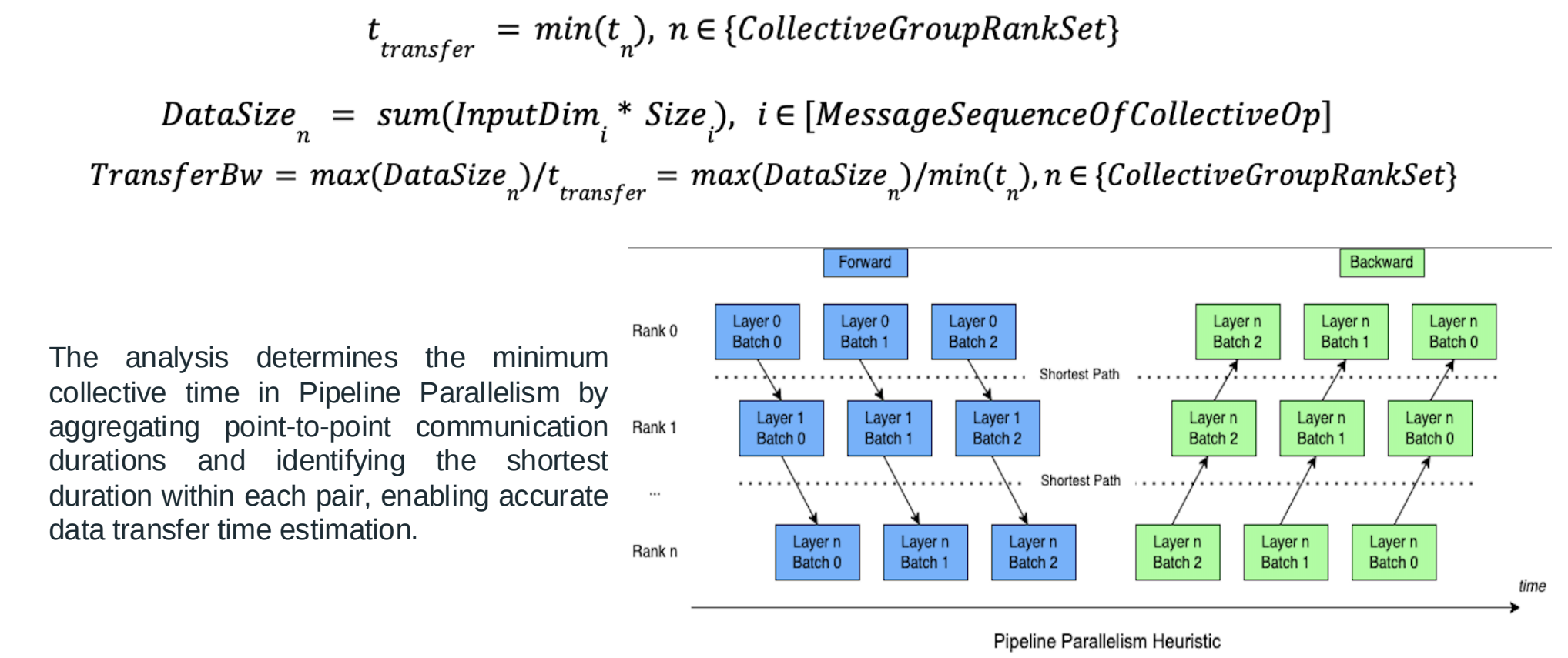
Collective Communication Analysis

A new hierarchical analysis accurately calculates exposed communication time in N-dimensional training, considering overlapping time from various parallelism techniques. This approach provides a comprehensive understanding of communication time, enabling users to optimize training workflows and improve performance.



Headroom Analysis

For scalable Large Language Model (LLM) training, estimating communication bandwidth headroom is crucial. However, current PyTorch traces make it challenging due to collective communication dynamics. Our proposed heuristic approach focuses on minimizing data transfer time, enabling accurate estimation of data transfer and waiting times, and providing a realistic assessment of communication bandwidth headroom for improved scalability and performance.



GPU Kernel Breakdown

The breakdown feature provides a detailed analysis of individual GPU performance by categorizing data into groups, such as duration, communication, computation, and specific operator types. This allows for a thorough understanding of kernel performance and helps users identify performance outliers and detect abnormal hardware in large-scale jobs.

